Filière de Big Data et Intelligence Artificielle
École Nationale des Sciences Appliquées de Tétouan

Credit Card Fraud Detection



Groupe :
EL ALAOUI SALEH Zouhair
GHOULAM Nouhaila
BENSALAH Aya
PROBI Nouhaila

Encadré par :
Pr. ATTAOUI Naoual

2025-2026

Table des matières

1 Introduction générale **5**
- 1.1 Contexte du projet . . . 5
- 1.2 Problématique . . . 5
- 1.3 Objectifs du projet . . . 5
- 1.4 Importer les bibliothèques . . . 6

2 Présentation du dataset **7**
- 2.1 Description générale . . . 7
- 2.2 Source des données . . . 7
- 2.3 Structure du dataset . . . 7
- 2.4 Description des variables . . . 8
- 2.5 Nature du problème . . . 8

3 Analyse exploratoire des données EDA **9**
- 3.1 Analyse de la variable cible . . . 9
- 3.2 Analyse des corrélations . . . 10
- 3.3 Analyse des valeurs extrêmes (outliers) . . . 10

4 Prétraitement des données **12**
- 4.1 Nettoyage des données . . . 12
- 4.2 Normalisation / Standardisation . . . 12
- 4.3 Séparation des données (train / test) . . . 12
- 4.4 Justification des choix de prétraitement . . . 13
- 4.5 Techniques d'équilibrage . . . 13
  - 4.5.1 UnderSamp . . . 14
  - 4.5.2 SMOTE . . . 14

5 Modélisation **16**
- 5.1 Choix des modèles . . . 16
- 5.2 Régression logistique . . . 16
  - 5.2.1 Modèle . . . 16
  - 5.2.2 Évaluation du modèle . . . 17
- 5.3 K plus proches voisins (KNN) . . . 19
  - 5.3.1 Modèle . . . 19
  - 5.3.2 Évaluation du modèle . . . 19
- 5.4 Arbre de décision . . . 21
  - 5.4.1 Modèle . . . 21
  - 5.4.2 Évaluation du modèle . . . 21
- 5.5 Forêt aléatoire . . . 22
  - 5.5.1 Modèle . . . 22
  - 5.5.2 Évaluation du modèle . . . 22

5.6 Machine à vecteurs de support linéaire (LinearSVC) 24
5.6.1 Modèle 24
5.6.2 Évaluation du modèle 25
5.7 Naive Bayes 26
5.7.1 Modèle 26
5.7.2 Évaluation du modèle 26

6 Analyse comparative des résultats **29**
6.1 Comparaison des performances des modèles 29
6.2 Interprétation des résultats 30
6.3 Choix du meilleur modèle 30

7 Conclusion **32**
7.1 Résumé des résultats 32
7.2 Limites du projet 32
7.3 Perspectives d'amélioration 32

Liste des tableaux

5.1 Performances du modèle KNN 19
5.2 Rapport de classification du modèle KNN 20
5.3 Performances du modèle Arbre de décision 21
5.4 Rapport de classification du modèle Arbre de décision 21
5.5 Performances du modèle Random Forest 23
5.6 Rapport de classification du modèle Random Forest 23
5.7 Performances du modèle LinearSVC (après équilibrage) 25
5.8 Rapport de classification du modèle LinearSVC (après équilibrage) 25
5.9 Performances du modèle Naive Bayes 27
5.10 Rapport de classification du modèle Naive Bayes 27

Table des figures

3.1 Matrice de corrélation 10
3.2 Boxplots 11

4.1 Heatmap 12

5.1 Matrice de confusion de Régression logistique 18
5.2 Matrice de confusion de KNN 20
5.3 Matrice de confusion d'Arbre de décision 22
5.4 Matrice de confusion de Forêt aléatoire 24
5.5 Matrice de confusion de SVM 26
5.6 Matrice de confusion de Naive Bayes 28

6.1 Tableau comparatif des performance des modèles 29
6.2 Courbes Precision-recall 29

Chapitre 1

Introduction générale

1.1 Contexte du projet

Avec l'essor du commerce électronique et des paiements numériques, l'utilisation des cartes bancaires a considérablement augmenté au cours des dernières années. Cette évolution s'accompagne malheureusement d'une hausse des fraudes financières, qui représentent un enjeu majeur pour les institutions bancaires et les clients.

La détection de fraude par carte de crédit est devenue une problématique critique dans le domaine de la data science et du machine learning, car elle permet de prévenir les pertes financières et de sécuriser les transactions. Cependant, ce type de problème est particulièrement complexe en raison du volume élevé des données, de leur nature dynamique et du caractère rare des fraudes.

Dans ce contexte, les techniques de classification supervisée offrent des solutions efficaces pour identifier automatiquement les transactions suspectes. Ce projet s'inscrit donc dans une démarche d'analyse et de modélisation visant à détecter les fraudes à partir de données réelles.]

1.2 Problématique

Le principal défi de ce projet réside dans la détection des transactions frauduleuses parmi un très grand nombre de transactions normales. En effet, le dataset étudié est fortement déséquilibré, avec une proportion extrêmement faible de fraudes (moins de 1

Cette situation rend la tâche difficile pour les modèles de machine learning, qui peuvent être biaisés vers la classe majoritaire (transactions normales) et ignorer la classe minoritaire (fraudes). Par conséquent, une simple mesure comme l'accuracy devient peu pertinente, car un modèle peut obtenir un score élevé sans réellement détecter les fraudes.

La problématique centrale peut donc être formulée comme suit :

Comment concevoir un modèle capable de détecter efficacement les fraudes tout en limitant les faux positifs dans un contexte de données fortement déséquilibrées ?

1.3 Objectifs du projet

L'objectif principal de ce projet est de développer et comparer plusieurs modèles de machine learning afin d'identifier la meilleure approche pour la détection de fraude.

Plus précisément, les objectifs sont :

-Analyser et comprendre le dataset de transactions bancaires

-Identifier les caractéristiques importantes pour la détection de fraude

-Appliquer des techniques de prétraitement adaptées, notamment pour gérer le déséquilibre des classes Implémenter plusieurs modèles de classification (régression logistique, KNN, arbre de décision, Random Forest, SVM, Naive Bayes)

-Évaluer les performances des modèles à l'aide de métriques adaptées (precision, recall, F1-score, AUC) Comparer les résultats afin de sélectionner le modèle le plus performant

L'objectif final est d'obtenir un modèle capable de détecter un maximum de fraudes tout en minimisant les erreurs de classification.

1.4 Importer les bibliothèques

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import math

#Normalisation
from sklearn.preprocessing import StandardScaler

#Les modeles
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split

#Metriques d'evaluation
from sklearn.metrics import (
accuracy_score,
precision_score,
recall_score,
f1_score,
roc_auc_score,
confusion_matrix,
classification_report
)
from sklearn.metrics import ConfusionMatrixDisplay

#validation croisee
from sklearn.model_selection import StratifiedKFold,
   cross_val_score
from sklearn.pipeline import Pipeline

#Equilibrage
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
```

Chapitre 2

Présentation du dataset

2.1 Description générale

Le dataset contient des transactions par carte de crédit réalisées en septembre 2013 par des titulaires européens. Il couvre deux jours d'activité, avec 492 fraudes sur 284 807 transactions, soit un fort déséquilibre (0,172% de fraudes).

Les variables sont uniquement numériques et résultent d'une transformation par PCA : $V1$ à $V28$ sont des composantes principales, tandis que *Time* (temps écoulé) et *Amount* (montant) n'ont pas été transformées. La variable cible *Class* vaut 1 en cas de fraude et 0 sinon.

En raison du déséquilibre des classes, des métriques adaptées comme précision-rappel sont recommandées, l'accuracy étant peu pertinente dans ce contexte.

2.2 Source des données

Le dataset utilisé dans ce projet provient de la plateforme Kaggle, une référence dans le partage de jeux de données pour le machine learning. Il s'agit du dataset *Credit Card Fraud Detection*, disponible au lien suivant : https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud. Ce jeu de données est largement utilisé dans la littérature pour l'étude des problèmes de classification déséquilibrée et de détection de fraude.

2.3 Structure du dataset

Le dataset contient 284 807 lignes et 31 colonnes : chaque ligne correspond à une observation (transaction) et chaque colonne à une variable (features et variable cible).

La majorité des variables sont de type `float64` (30 colonnes), ce qui indique des données numériques continues, tandis que la variable *Class* est de type `int64`, représentant la variable cible (classification binaire). On retrouve des variables nommées $V1$ à $V28$, ainsi que *Time* et *Amount*, déjà transformées par PCA.

Toutes les colonnes du dataset sont numériques, incluant *Time*, $V1$ à $V28$, *Amount* et *Class*. Aucune variable de type `object` n'est présente, ce qui signifie qu'il n'y a pas de données catégorielles à encoder. Le dataset est donc directement exploitable pour les modèles de machine learning sans étape d'encodage supplémentaire.

2.4 Description des variables

Les variables V1 à V28 sont le résultat d'une Analyse en Composantes Principales (PCA) appliquée en amont sur les données brutes. Cette technique transforme les variables originales en composantes orthogonales et indépendantes, pour deux raisons principales : anonymiser les informations sensibles des clients, et réduire le bruit en ne conservant que les dimensions les plus informatives. C'est pourquoi ces variables apparaissent centrées en zéro avec un écart-type unitaire dans les statistiques descriptives. Seules Amount et Class conservent leurs valeurs originales et n'ont pas subi cette transformation.

Les variables *Time*, *Amount* et *Class* n'ont pas été transformées par la PCA et conservent donc leur signification initiale. La variable *Time* représente le temps écoulé en secondes entre chaque transaction et la première transaction du dataset. La variable *Amount* correspond au montant de la transaction, fournissant une information importante sur la valeur financière de chaque opération. Enfin, la variable *Class* est la variable cible : elle indique si une transaction est frauduleuse (1) ou non (0). Ces variables complètent les composantes principales en apportant des informations directement interprétables.

2.5 Nature du problème

Le problème étudié est un problème de **classification binaire**, dont l'objectif est de prédire si une transaction est frauduleuse ou non. La variable cible *Class* prend deux valeurs possibles : 0 pour une transaction normale et 1 pour une transaction frauduleuse.

Ce type de problème consiste à apprendre une fonction de décision capable de distinguer ces deux classes à partir des variables explicatives. Dans ce contexte, chaque observation est associée à une seule classe.

Cependant, le dataset est fortement déséquilibré, avec une proportion très faible de transactions frauduleuses. Cela rend la tâche plus complexe et nécessite l'utilisation de techniques adaptées, tant au niveau du prétraitement que de l'évaluation des modèles, afin de garantir une bonne détection de la classe minoritaire.

Chapitre 3

Analyse exploratoire des données EDA

3.1 Analyse de la variable cible

```
df["Class"].value_counts()
```

Class
0 284315
1 492
Name : count, dtype : int64

La variable cible est fortement déséquilibrée : la classe 0 (non fraude) contient 284 315 observations, tandis que la classe 1 (fraude) ne contient que 492 observations. Cela représente une proportion extrêmement faible de la classe minoritaire, ce qui peut biaiser les modèles vers la classe majoritaire et rendre l'accuracy trompeuse, d'où la nécessité d'utiliser des techniques ou des métriques adaptées.

3.2 Analyse des corrélations

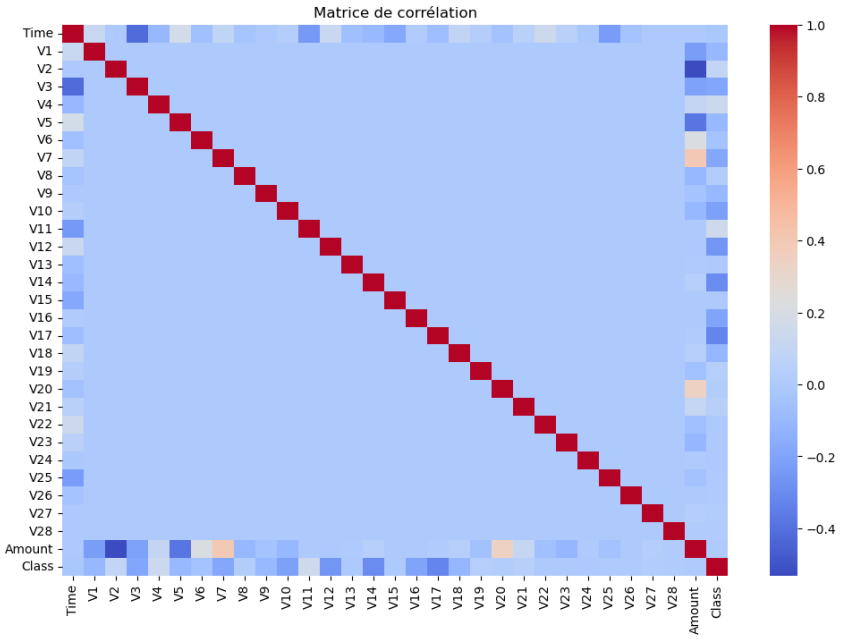


FIGURE 3.1 – Matrice de corrélation

La matrice de corrélation montre que la majorité des variables (V1 à V28) sont très faiblement corrélées entre elles, ce qui est cohérent avec des données transformées (probablement par PCA). On observe également que la variable Class n'a pas de corrélation forte avec une seule variable spécifique, ce qui indique que la détection de fraude dépend d'une combinaison de plusieurs features plutôt que d'un seul facteur. La variable Amount présente quelques corrélations faibles avec certaines variables, tandis que Time est globalement peu corrélée avec les autres. Globalement, l'absence de fortes corrélations suggère un faible risque de multicolinéarité et confirme que le problème nécessite des modèles capables de capter des relations complexes entre variables.

3.3 Analyse des valeurs extrêmes (outliers)

Les variables dans ce dataset présentent un nombre important d'outliers, notamment pour V8, V20, V27, V28 et Amount, ce qui indique une forte dispersion des données. Certaines variables comme V11 ou Time contiennent peu ou pas d'outliers, mais globalement le dataset est marqué par des valeurs extrêmes fréquentes. Cela est cohérent avec un problème de détection de fraude, où les transactions frauduleuses se distinguent souvent par des comportements atypiques. La présence de nombreux outliers peut influencer certains modèles sensibles (comme KNN ou SVM) et nécessite éventuellement des techniques de normalisation ou des modèles robustes.

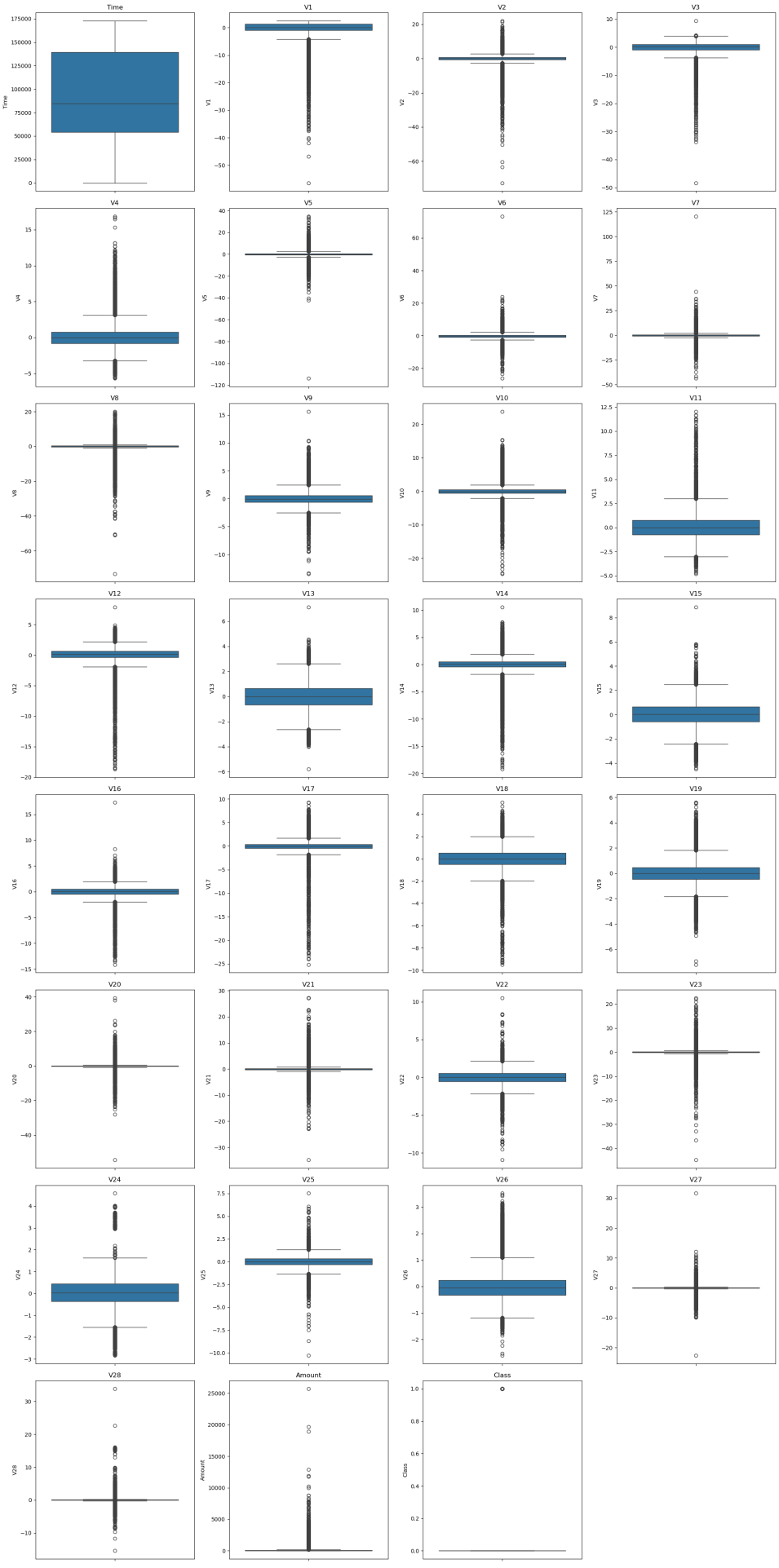


FIGURE 3.2 – Boxplots

Chapitre 4

Prétraitement des données

4.1 Nettoyage des données

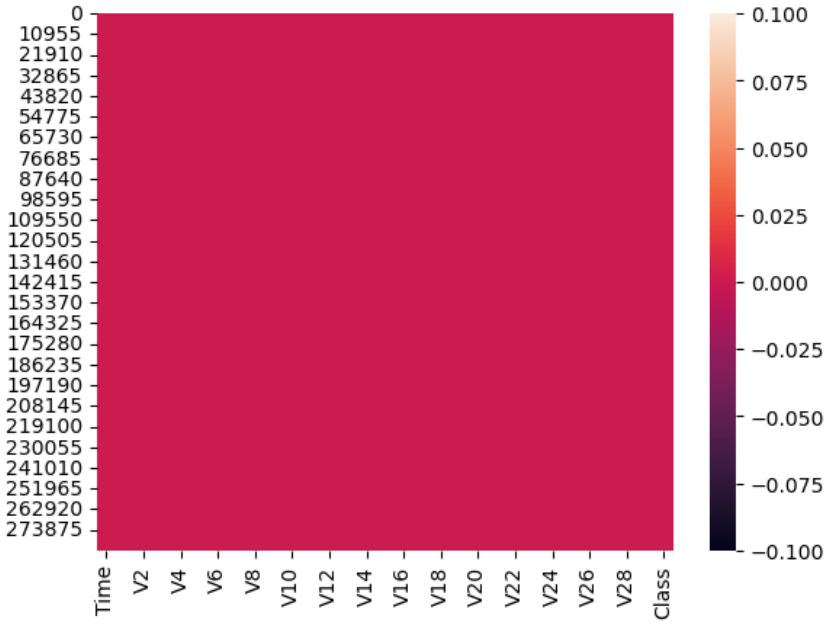


FIGURE 4.1 – Heatmap

La visualisation confirme que le dataset est propre et bien structuré, sans anomalies apparentes nécessitant un nettoyage supplémentaire. On n'observe ni valeurs manquantes, ni incohérences, ni variables problématiques. Les données sont homogènes, entièrement numériques et déjà préparées (notamment les variables V1 à V28). Ainsi, aucune étape de nettoyage n'est nécessaire, et le dataset peut être directement utilisé pour l'entraînement des modèles, en se concentrant plutôt sur des étapes comme la normalisation et l'équilibrage des classes.

4.2 Normalisation / Standardisation

La normalisation va être faite dans les pipelines des modèles.

4.3 Séparation des données (train / test)

```
x = df[['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', '
    V10','V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18','
    V19','V20','V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27','V28
    ', 'Amount']]

y = df['Class']

x_train, x_test, y_train, y_test = train_test_split(x, y,
    test_size=0.2, random_state=42,stratify=y)
print("taille d'enssemble d'entrainement est : " ,x_train.shape)
print("taille d'enssemble de test est : " ,x_test.shape)
```

taille d'enssemble d'entrainement est : (227845, 29)
taille d'enssemble de test est : (56962, 29)

Les variables explicatives X sont constituées des features $V1$ à $V28$ ainsi que $Amount$, tandis que la variable cible y correspond à $Class$. Les données ont ensuite été séparées en deux ensembles : 80% pour l'entraînement et 20% pour le test, en utilisant un échantillonnage stratifié afin de conserver la proportion des classes. L'ensemble d'entraînement contient 227 845 observations avec 29 variables, tandis que l'ensemble de test contient 56 962 observations avec 29 variables, garantissant une évaluation fiable des performances du modèle sur des données non vues.

4.4 Justification des choix de prétraitement

Le choix d'appliquer un échantillonnage stratifié lors de la séparation des données se justifie par le fort déséquilibre des classes dans le dataset. En effet, sans stratification, il est possible que certaines partitions contiennent une proportion très faible, voire nulle, de la classe minoritaire, ce qui biaiserait l'apprentissage et l'évaluation des modèles. Le split en 80% pour l'entraînement et 20% pour le test constitue un compromis classique permettant de disposer de suffisamment de données pour entraîner le modèle tout en conservant un échantillon indépendant pour évaluer sa capacité de généralisation. Enfin, l'utilisation du paramètre `random_state=42` garantit la reproductibilité des résultats, ce qui est essentiel dans une démarche expérimentale.

4.5 Techniques d'équilibrage

Dans un dataset de détection de fraude comme celui des cartes bancaires, les classes sont fortement déséquilibrées, avec une grande majorité de transactions normales et une très faible proportion de fraudes. Dans ce contexte, un modèle peut facilement obtenir une accuracy élevée simplement en prédisant toujours la classe majoritaire, sans réellement apprendre à détecter les fraudes. Cela rend les performances trompeuses et surtout inefficaces dans un cadre réel, où l'objectif principal est d'identifier les cas rares mais critiques. L'équilibrage des données devient donc indispensable pour permettre au modèle d'accorder suffisamment d'importance à la classe minoritaire, améliorer le recall des fraudes et construire un classifieur plus robuste et pertinent face à ce type de problème.

4.5.1 UnderSamp

L'undersampling est une technique d'équilibrage des classes qui consiste à réduire le nombre d'observations de la classe majoritaire afin de rétablir un meilleur équilibre avec la classe minoritaire. Contrairement aux méthodes de suréchantillonnage comme SMOTE, aucune nouvelle donnée n'est générée : certaines observations existantes sont simplement supprimées. Cette approche permet de diminuer le biais du modèle envers la classe dominante, mais elle peut entraîner une perte d'information si des données importantes sont retirées. Par exemple, supposons que dans un dataset de détection de fraude, on dispose d'un grand nombre d'observations appartenant à la classe majoritaire (non fraude) et de peu d'observations de la classe minoritaire (fraude). Considérons quelques points de la classe majoritaire :

$$A = (V1 = 1.0,\ V2 = 1.5)$$

$$B = (V1 = 1.2,\ V2 = 1.7)$$

$$C = (V1 = 0.9,\ V2 = 1.4)$$

La méthode d'*undersampling* consiste à réduire le nombre d'observations de la classe majoritaire en sélectionnant un sous-ensemble de ces données. Par exemple, on peut conserver seulement deux observations parmi les trois :

$$\{A, B, C\} \longrightarrow \{A, C\}$$

Ainsi, certaines observations (ici B) sont supprimées afin de rééquilibrer le dataset. Contrairement à SMOTE, aucune nouvelle donnée n'est créée ; on diminue simplement le nombre d'exemples de la classe dominante, ce qui permet d'éviter que le modèle soit biaisé vers cette classe.

```
rus = RandomUnderSampler(random_state=42)

X_train_res, y_train_res = rus.fit_resample(x_train, y_train)

print("Avant :", y_train.value_counts())
print("Apr s :", y_train_res.value_counts())
```

```
Avant : Class
0    227451
1       394
Name: count, dtype: int64
Après : Class
0    394
1    394
Name: count, dtype: int64
```

4.5.2 SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) est une méthode d'équilibrage des classes utilisée lorsque le dataset est fortement déséquilibré. Contrairement au simple suréchantillonnage qui duplique les observations de la classe minoritaire, SMOTE génère de nouvelles données synthétiques en interpolant entre des exemples proches de cette

classe. Concrètement, pour une observation minoritaire, l'algorithme sélectionne ses voisins les plus proches et crée de nouveaux points situés entre eux, ce qui permet d'enrichir l'espace des données de manière plus réaliste. Par exemple, dans un problème de détection de fraude où la classe frauduleuse est très rare, SMOTE permet de générer artificiellement de nouvelles transactions frauduleuses similaires aux existantes, afin de rééquilibrer le dataset et d'améliorer la capacité du modèle à détecter ces cas rares. Par exemple, supposons que dans un dataset de détection de fraude, on dispose de deux observations appartenant à la classe minoritaire (fraude) :

$$A = (V1 = 2.0,\ V2 = 3.0)$$

$$B = (V1 = 2.4,\ V2 = 3.6)$$

La méthode SMOTE génère une nouvelle observation synthétique en interpolant entre ces deux points. Cette interpolation est donnée par :

$$x_{\text{new}} = x_i + \lambda \times (x_j - x_i), \quad \lambda \in [0, 1]$$

Par exemple, pour $\lambda = 0.5$:

$$x_{\text{new}} = (2.0, 3.0) + 0.5 \times ((2.4, 3.6) - (2.0, 3.0))$$

$$x_{\text{new}} = (2.0, 3.0) + 0.5 \times (0.4, 0.6)$$

$$x_{\text{new}} = (2.2, 3.3)$$

Ainsi, une nouvelle observation synthétique est créée entre deux points existants de la classe minoritaire, ce qui permet d'enrichir le dataset sans dupliquer les données.

```
smote = SMOTE(random_state=42)

X_train_res, y_train_res = smote.fit_resample(x_train, y_train)

print("Avant :", y_train.value_counts())
print("Apr s :", y_train_res.value_counts())
```

```
Avant : Class
0    227451
1       394
Name: count, dtype: int64

Après : Class
0    227451
1    227451
Name: count, dtype: int64
```

Chapitre 5

Modélisation

Nous allons nous concentrer sur la modélisation en appliquant différents algorithmes de classification. Afin d'obtenir des résultats plus fiables et pertinents, les modèles seront entraînés sur des données préalablement équilibrées. Cette étape est essentielle dans le contexte de la détection de fraude, où la classe minoritaire doit être correctement représentée pour améliorer la capacité des modèles à identifier les cas frauduleux. Par ailleurs, en raison du fort déséquilibre du dataset, la métrique d'accuracy n'est pas fiable dans ce contexte, car elle peut donner une impression trompeuse de bonne performance même si les fraudes ne sont pas correctement détectées. Il est donc nécessaire de privilégier des métriques plus adaptées telles que le recall, la précision, le F1-score et l'AUC pour évaluer les modèles.

5.1 Choix des modèles

Plusieurs modèles de classification supervisée ont été sélectionnés afin de comparer leurs performances sur le problème de détection de fraude. Le choix s'est porté sur des algorithmes variés afin d'évaluer différents comportements face aux données équilibrées. Cette comparaison permet d'identifier le modèle le plus adapté en termes de performance et de capacité à détecter la classe minoritaire.

5.2 Régression logistique

La régression logistique constitue un premier modèle de référence dans ce projet. Elle est largement utilisée en classification binaire grâce à sa simplicité, sa rapidité d'entraînement et sa capacité à fournir des probabilités interprétables. Dans le contexte de la détection de fraude, elle permet d'établir une première base de comparaison avant d'explorer des modèles plus complexes.

5.2.1 Modèle

```
pipeline_1_equ = Pipeline([
    ("smote", SMOTE(random_state=42)),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

```
pipeline_1_equ.fit(x_train, y_train)
```

5.2.2 Évaluation du modèle

Validation croisée

```
# Validation crois e stratifi e (5 folds)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# recall sur chaque fold
scores = cross_val_score(
    pipeline_1_equ,
    x_train, y_train,
    cv=cv,
    scoring='recall'
)

print("Scores par fold :", scores)
print("recall moyen :", scores.mean())
print(" cart -type :", scores.std())
```

Scores par fold : [0.92307692 0.87341772 0.87341772 0.92405063 0.88607595]
recall moyen : 0.8960077896786757
Écart-type : 0.022971298833111033

Les résultats de la validation croisée montrent que le modèle atteint un rappel moyen de 0.896, avec un écart-type de 0.023. Cela indique que le modèle est globalement performant pour la détection des cas positifs (fraudes), tout en présentant une bonne stabilité entre les différents folds. En effet, la faible valeur de l'écart-type traduit une variabilité limitée des performances, ce qui signifie que le modèle généralise correctement sur différents sous-ensembles de données. Ainsi, le modèle offre un compromis satisfaisant entre efficacité de détection et robustesse, sans fluctuations importantes d'un fold à l'autre.

Mesures de performance

```
y_pred1_equ = pipeline_1_equ.predict(x_test)
y_prob1_equ = pipeline_1_equ.predict_proba(x_test)[:, 1]
```

```
print("===== Logistic Regression ==============================")
print("Accuracy  :", accuracy_score(y_test, y_pred1_equ))
print("Precision :", precision_score(y_test, y_pred1_equ))
print("Recall    :", recall_score(y_test, y_pred1_equ))
print("F1-score  :", f1_score(y_test, y_pred1_equ))
print("AUC       :", roc_auc_score(y_test, y_prob1_equ))

print("\nRapport :==============================================")
print(classification_report(y_test, y_pred1_equ))
```

```
===== Logistic Regression ==============================
Accuracy  : 0.9811979916435518
Precision : 0.07732406602953953
Recall    : 0.9081632653061225
```

```
F1-score  : 0.14251401120896717
AUC       : 0.9720373817084516

Rapport :==============================================
              precision    recall  f1-score   support

           0       1.00      0.98      0.99     56864
           1       0.08      0.91      0.14        98

    accuracy                           0.98     56962
   macro avg       0.54      0.94      0.57     56962
weighted avg       1.00      0.98      0.99     56962
```

Sur l'ensemble du test, le modèle de régression logistique atteint une accuracy élevée de 0.98, ce qui peut donner l'impression d'une très bonne performance globale. Toutefois, cette métrique est peu pertinente dans un contexte de données déséquilibrées. Le rappel est élevé (0.91), indiquant que le modèle parvient à détecter la majorité des fraudes. En revanche, la précision est très faible (0.08), ce qui signifie qu'une grande proportion des transactions prédites comme frauduleuses sont en réalité légitimes, générant ainsi de nombreuses fausses alertes.

Le F1-score (0.14) reste faible, confirmant un déséquilibre important entre précision et rappel. Malgré un AUC élevé (0.97), qui montre une bonne capacité de séparation entre les classes, le modèle n'est pas suffisamment fiable en pratique à cause de son taux élevé de faux positifs. Une amélioration est donc nécessaire afin de mieux équilibrer la précision et le rappel.

Matrice de confusion

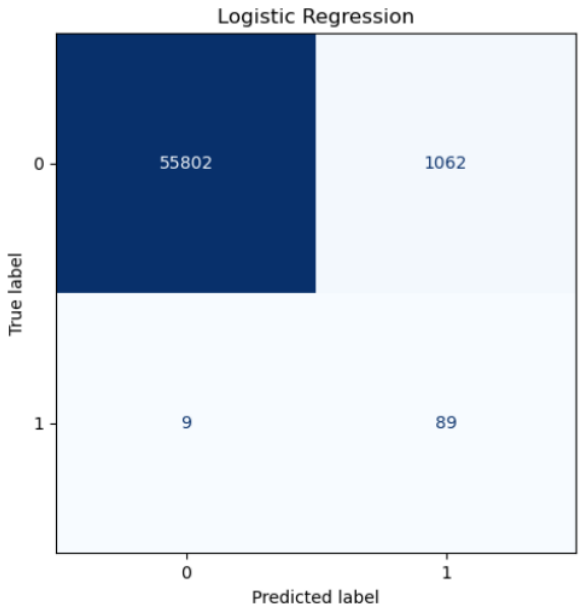


FIGURE 5.1 – Matrice de confusion de Régression logistique

La matrice de confusion confirme cette tendance : sur 98 fraudes réelles, 89 sont correctement détectées, contre seulement 9 non détectées. En revanche, 1062 transactions

normales sont classées à tort comme frauduleuses. Cela montre que le modèle privilégie fortement la détection des fraudes, mais au prix d'un grand nombre de fausses alertes.

5.3 K plus proches voisins (KNN)

Le modèle KNN (K plus proches voisins) est un algorithme basé sur la distance qui classe une observation en fonction des k observations les plus proches dans l'espace des variables.

5.3.1 Modèle

```
model2 = KNeighborsClassifier(n_neighbors=5)
```

5.3.2 Évaluation du modèle

Validation croisée

```
Scores par fold : [0.80769231 0.7721519  0.79746835 0.84810127 0.86075949]
Recall moyen : 0.8172346640701071
Écart-type : 0.03274561707430257
```

Les résultats de la validation croisée montrent que le modèle atteint un recall moyen de 0.817, indiquant qu'il détecte une proportion importante des fraudes. Les scores obtenus sur les différents folds restent relativement proches, avec un écart-type de 0.033, ce qui traduit une stabilité correcte du modèle. Cette variabilité modérée montre que les performances ne fluctuent pas fortement selon les sous-ensembles de données, bien qu'elles soient légèrement moins homogènes. Globalement, le modèle présente une capacité de généralisation satisfaisante, mais une amélioration pourrait être envisagée afin d'augmenter le recall tout en réduisant davantage les variations entre les folds.

Mesures de performance

Métrique	Valeur
Accuracy	0.9976
Precision	0.4059
Recall	0.8367
F1-score	0.5467
AUC	0.9228

Table 5.1 – Performances du modèle KNN

Classe	Precision	Recall	F1-score	Support
0	1.00	1.00	1.00	56864
1	0.41	0.84	0.55	98
Accuracy		1.00		56962
Macro avg	0.70	0.92	0.77	56962
Weighted avg	1.00	1.00	1.00	56962

TABLE 5.2 – Rapport de classification du modèle KNN

Les performances du modèle KNN montrent une accuracy très élevée de 0.9976, mais cette métrique reste peu significative dans un contexte de données déséquilibrées. Le modèle obtient un recall de 0.8367, ce qui indique qu'il parvient à détecter une proportion importante des fraudes, bien que certaines restent non identifiées. La précision de 0.4059 est nettement meilleure que celle de certains modèles précédents, ce qui signifie que les prédictions de fraude sont plus fiables, avec moins de faux positifs.

Le F1-score de 0.5467 reflète un compromis plus équilibré entre précision et rappel. De plus, l'AUC de 0.9228 confirme une bonne capacité de discrimination entre les classes. Le rapport de classification montre également que la classe majoritaire (0) est parfaitement reconnue, tandis que la classe minoritaire (1) reste plus difficile à prédire, avec un rappel de 0.84. Globalement, le modèle KNN offre un meilleur équilibre entre détection des fraudes et réduction des fausses alertes, ce qui le rend plus pertinent dans ce contexte.

Matrice de confusion

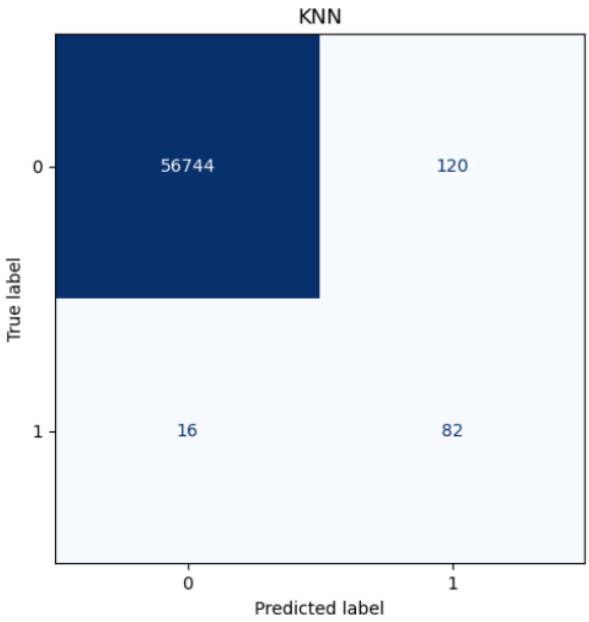


FIGURE 5.2 – Matrice de confusion de KNN

La matrice de confusion montre que 82 fraudes sur 98 sont correctement détectées, tandis que 16 ne sont pas détectées. Le nombre de faux positifs reste modéré, ce qui traduit une bonne qualité globale de classification.

5.4 Arbre de décision

L'arbre de décision est un modèle qui segmente les données en appliquant des règles de décision successives afin de classer les observations.

5.4.1 Modèle

```
model3 = DecisionTreeClassifier(
    random_state=42
)
```

5.4.2 Évaluation du modèle

Validation croisée

```
Scores par fold : [0.82051282 0.65822785 0.75949367 0.84810127 0.79746835]
Recall moyen : 0.7767607919506654
Écart-type : 0.06600108878937917
```

Les résultats de la validation croisée montrent que le modèle atteint un recall moyen de 0.777, indiquant une capacité correcte à détecter les fraudes, bien que certaines ne soient pas identifiées. Les scores obtenus sur les différents folds présentent une certaine variabilité, avec un écart-type de 0.066, ce qui traduit une stabilité modérée du modèle. Cette dispersion plus importante par rapport à d'autres modèles suggère que les performances dépendent davantage des sous-ensembles de données utilisés. Ainsi, bien que le modèle reste globalement efficace, une amélioration serait nécessaire afin d'augmenter le recall et de renforcer la robustesse du modèle.

Mesures de performance

Métrique	**Valeur**
Accuracy	0.9982
Precision	0.4848
Recall	0.8163
F1-score	0.6084
AUC	0.9074

TABLE 5.3 – Performances du modèle Arbre de décision

Classe	**Precision**	**Recall**	**F1-score**	**Support**
0	1.00	1.00	1.00	56864
1	0.48	0.82	0.61	98
Accuracy	1.00			56962
Macro avg	0.74	0.91	0.80	56962
Weighted avg	1.00	1.00	1.00	56962

TABLE 5.4 – Rapport de classification du modèle Arbre de décision

Sur les données de test, le modèle atteint une accuracy de 0.9982. Le recall est de 0.82, ce qui signifie que 82% des fraudes sont détectées. La précision est de 0.48, indiquant qu'environ la moitié des prédictions de fraude sont correctes. Le F1-score de 0.61 traduit un bon équilibre global.

Matrice de confusion

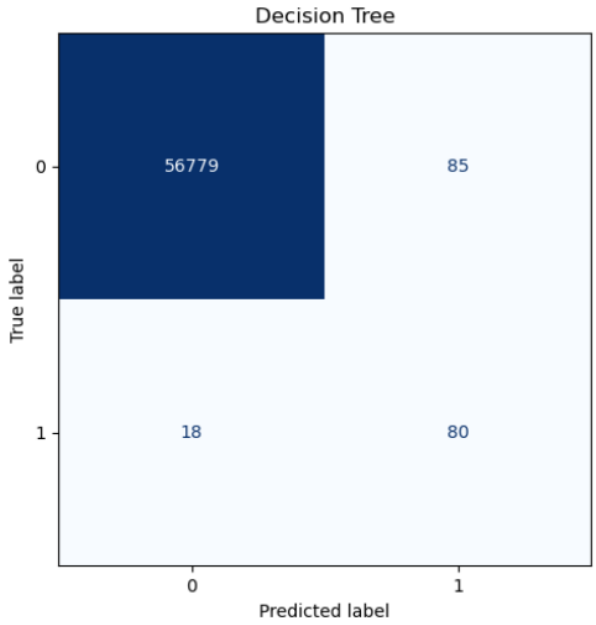


FIGURE 5.3 – Matrice de confusion d'Arbre de décision

La matrice de confusion montre que 80 fraudes sur 98 sont correctement détectées, avec 18 fraudes non détectées. Le nombre de faux positifs reste présent mais maîtrisé, ce qui confirme une performance globale satisfaisante.

5.5 Forêt aléatoire

La forêt aléatoire est un ensemble d'arbres de décision combinés pour améliorer les performances.

5.5.1 Modèle

```
model4 = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
```

5.5.2 Évaluation du modèle

Validation croisée

```
Scores par fold : [0.84615385 0.73417722 0.75949367 0.88607595 0.83544304]
Recall moyen : 0.8122687439143135
```

```
Écart-type : 0.05659725581224602
```

Les résultats de la validation croisée indiquent que le modèle atteint un recall moyen de 0.812, ce qui traduit une bonne capacité à détecter les fraudes. Les scores obtenus sur les différents folds restent globalement cohérents, avec un écart-type de 0.056, indiquant une stabilité correcte du modèle. Toutefois, cette variabilité modérée montre que les performances peuvent légèrement fluctuer selon les sous-ensembles de données. Ainsi, le modèle présente une bonne capacité de généralisation, bien qu'une amélioration soit envisageable afin d'augmenter davantage le recall et de réduire les variations observées.

Mesures de performance

Métrique	**Valeur**
Accuracy	0.9995
Precision	0.8469
Recall	0.8469
F1-score	0.8469
AUC	0.9680

Table 5.5 – Performances du modèle Random Forest

Classe	**Precision**	**Recall**	**F1-score**	**Support**
0	1.00	1.00	1.00	56864
1	0.85	0.85	0.85	98
Accuracy	1.00			56962
Macro avg	0.92	0.92	0.92	56962
Weighted avg	1.00	1.00	1.00	56962

Table 5.6 – Rapport de classification du modèle Random Forest

Sur les données de test, le modèle atteint une accuracy de 0.9995. La précision, le recall et le F1-score sont tous proches de 0.85, ce qui signifie que le modèle détecte correctement environ 85% des fraudes tout en maintenant un bon niveau de précision. L'AUC élevée (0.968) confirme la très bonne capacité de discrimination du modèle.

Matrice de confusion

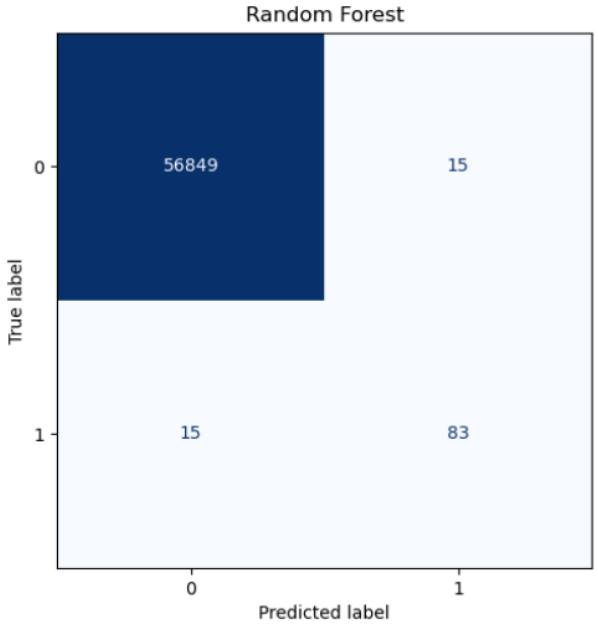


FIGURE 5.4 – Matrice de confusion de Forêt aléatoire

La matrice de confusion montre que 83 fraudes sur 98 sont correctement détectées, avec seulement 15 non détectées. Le nombre de faux positifs est très faible, ce qui indique une excellente qualité de classification.

5.6 Machine à vecteurs de support linéaire (LinearSVC)

Le modèle SVM linéaire (Support Vector Machine) est un algorithme de classification qui cherche à construire une frontière de décision linéaire optimale séparant les différentes classes. Il repose sur le principe de maximisation de la marge entre les points les plus proches de chaque classe, appelés vecteurs de support.

Dans le cas de LinearSVC, une version optimisée pour les grands ensembles de données, le modèle est particulièrement efficace lorsque les données sont linéairement séparables ou proches de l'être. Il est également adapté aux problèmes de grande dimension, comme la détection de fraude.

Cependant, dans un contexte de données déséquilibrées, comme celui de ce projet, le modèle a tendance à favoriser le rappel (recall) au détriment de la précision, ce qui peut entraîner un nombre élevé de faux positifs.

5.6.1 Modèle

```
model5 = LinearSVC(
    random_state=42,
    dual='auto'
)
```

5.6.2 Évaluation du modèle

Validation croisée

```
Scores par fold : [0.92307692 0.86075949 0.86075949 0.93670886 0.87341772]
Recall moyen : 0.8909444985394354
Écart-type : 0.0324232216370281
```

Les résultats de la validation croisée montrent que le modèle LinearSVC atteint un recall moyen de 0.891, indiquant une très bonne capacité à détecter les fraudes. Les scores obtenus sur les différents folds sont relativement proches, avec un écart-type de 0.032, ce qui traduit une bonne stabilité du modèle. Cette faible variabilité montre que le modèle généralise correctement sur différents sous-ensembles de données.

Mesures de performance

Métrique	**Valeur**
Accuracy	0.9843
Precision	0.0915
Recall	0.9082
F1-score	0.1662
AUC	0.9734

TABLE 5.7 – Performances du modèle LinearSVC (après équilibrage)

Classe	**Precision**	**Recall**	**F1-score**	**Support**
0	1.00	0.98	0.99	56864
1	0.09	0.91	0.17	98
Accuracy	0.98			56962
Macro avg	0.55	0.95	0.58	56962
Weighted avg	1.00	0.98	0.99	56962

TABLE 5.8 – Rapport de classification du modèle LinearSVC (après équilibrage)

Sur l'ensemble du test, le modèle LinearSVC atteint une accuracy élevée de 0.9843, mais cette métrique reste peu pertinente dans un contexte de données déséquilibrées. Le modèle présente un recall élevé de 0.9082, confirmant sa capacité à détecter la majorité des fraudes.

Cependant, la précision est très faible (0.0915), ce qui signifie qu'un grand nombre de transactions normales sont incorrectement classées comme frauduleuses. Cela se traduit par un F1-score faible (0.1662), révélant un déséquilibre important entre précision et rappel.

Malgré un AUC élevé (0.9731), indiquant une bonne capacité de séparation entre les classes, le modèle génère trop de faux positifs pour être fiable en pratique. Une amélioration est donc nécessaire afin de mieux équilibrer la précision et le recall.

Matrice de confusion

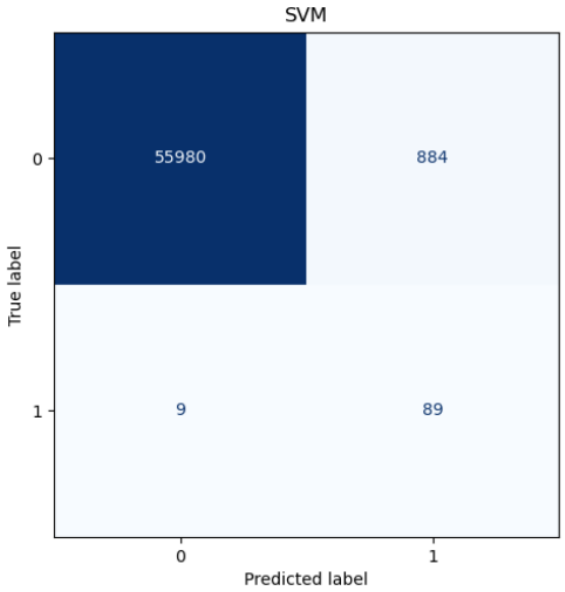


FIGURE 5.5 – Matrice de confusion de SVM

La matrice de confusion montre que 89 fraudes sur 98 sont correctement détectées, mais un grand nombre de transactions normales sont classées à tort comme frauduleuses, ce qui limite la fiabilité du modèle.

5.7 Naive Bayes

Le modèle Naive Bayes est un classificateur probabiliste basé sur le théorème de Bayes, qui suppose l'indépendance conditionnelle des variables explicatives. Dans ce travail, nous utilisons la variante Gaussian Naive Bayes (GaussianNB), adaptée aux variables continues, où les données sont supposées suivre une distribution normale.

Ce modèle est particulièrement apprécié pour sa simplicité, sa rapidité d'entraînement et sa capacité à fonctionner efficacement même sur des ensembles de données de grande taille. Toutefois, l'hypothèse d'indépendance entre les variables peut être restrictive et influencer les performances, notamment dans des problèmes complexes comme la détection de fraude.

5.7.1 Modèle

```
model6 = GaussianNB()
```

5.7.2 Évaluation du modèle

Validation croisée

```
Scores par fold : [0.8974359  0.79746835 0.82278481 0.93670886 0.82278481]
Recall moyen : 0.8554365465757872
Écart-type : 0.05265304855817806
```

Les résultats de la validation croisée montrent que le modèle Naive Bayes atteint un recall moyen de 0.855, indiquant une bonne capacité à détecter les fraudes. Les scores obtenus sur les différents folds restent relativement proches, avec un écart-type de 0.053, ce qui traduit une stabilité correcte du modèle. Cette variabilité modérée indique que le modèle généralise de manière satisfaisante sur différents sous-ensembles de données.

Mesures de performance

Métrique	**Valeur**
Accuracy	0.9729
Precision	0.0531
Recall	0.8776
F1-score	0.1002
AUC	0.9639

TABLE 5.9 – Performances du modèle Naive Bayes

Classe	**Precision**	**Recall**	**F1-score**	**Support**
0	1.00	0.97	0.99	56864
1	0.05	0.88	0.10	98
Accuracy	0.97			56962
Macro avg	0.53	0.93	0.54	56962
Weighted avg	1.00	0.97	0.98	56962

TABLE 5.10 – Rapport de classification du modèle Naive Bayes

Sur l'ensemble du test, le modèle Naive Bayes atteint une accuracy de 0.9729, mais cette métrique est peu pertinente dans un contexte de données déséquilibrées. Le modèle présente un recall élevé (0.8776), ce qui signifie qu'il détecte efficacement la majorité des fraudes.

Cependant, la précision est très faible (0.0531), indiquant qu'un grand nombre de transactions normales sont classées à tort comme frauduleuses. Cela entraîne un F1-score faible (0.1002), révélant un fort déséquilibre entre précision et rappel.

Malgré un AUC de 0.9639, qui montre une bonne capacité de discrimination entre les classes, le modèle souffre d'un nombre élevé de faux positifs. Par conséquent, il nécessite des ajustements pour améliorer la précision tout en conservant un bon niveau de recall.

Matrice de confusion

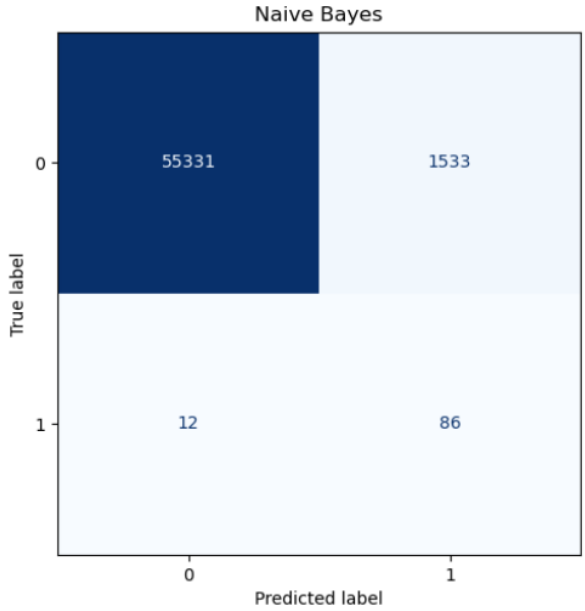


FIGURE 5.6 – Matrice de confusion de Naive Bayes

La matrice de confusion montre que 86 fraudes sur 98 sont détectées, mais un grand nombre de transactions normales sont classées comme frauduleuses, ce qui réduit fortement la qualité globale du modèle.

Chapitre 6

Analyse comparative des résultats

Dans ce chapitre, nous analysons et comparons les performances des différents modèles de classification appliqués au problème de détection de fraude. L'objectif est d'évaluer leur capacité à identifier correctement les transactions frauduleuses à travers plusieurs métriques, notamment la précision, le recall, le F1-score et l'AUC. Cette analyse permet de mieux comprendre le comportement de chaque modèle et d'identifier celui qui offre les meilleures performances globales.

6.1 Comparaison des performances des modèles

Modèle	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9812	0.0773	0.9082	0.1425	0.9720
KNN	0.9976	0.4059	0.8367	0.5467	0.9228
Decision Tree	0.9982	0.4848	0.8163	0.6084	0.9074
Random Forest	0.9995	0.8469	0.8469	0.8469	0.9680
SVM	0.9843	0.0915	0.9082	0.1662	0.9734
Naive Bayes	0.9729	0.0531	0.8776	0.1002	0.9639

FIGURE 6.1 – Tableau comparatif des performance des modèles

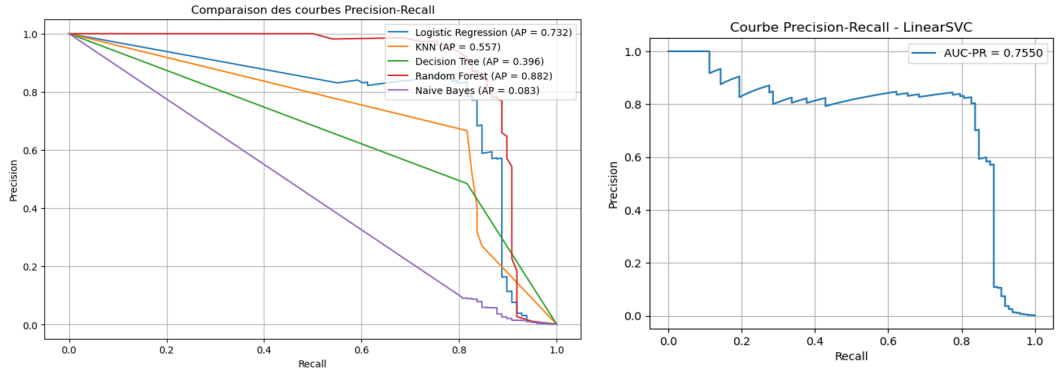


FIGURE 6.2 – Courbes Precision-recall

6.2 Interprétation des résultats

L'analyse du tableau comparatif montre des différences importantes entre les modèles en fonction des métriques utilisées. La régression logistique présente un recall très élevé (0.91), ce qui signifie qu'elle détecte la majorité des fraudes, mais sa précision reste très faible (0.077), ce qui entraîne un grand nombre de faux positifs. Le SVM présente un comportement similaire avec un recall de 0.91 mais une précision faible (0.09), traduisant une difficulté à distinguer correctement les transactions normales des fraudes.

Le modèle Naive Bayes affiche également un recall élevé (0.88), mais sa précision très faible (0.05) limite fortement sa fiabilité. Ces modèles ont donc tendance à privilégier la détection des fraudes au détriment de la précision.

Le modèle KNN montre une amélioration notable avec une précision de 0.41 et un recall de 0.84, ce qui traduit un meilleur équilibre entre la détection des fraudes et la réduction des erreurs. L'arbre de décision renforce cet équilibre avec une précision de 0.48 et un recall de 0.82, ainsi qu'un F1-score de 0.61, indiquant une meilleure gestion globale des deux classes.

Enfin, le modèle Random Forest se distingue par des performances élevées sur toutes les métriques : une précision de 0.85, un recall de 0.85 et un F1-score de 0.85. Cela signifie que le modèle est capable de détecter efficacement les fraudes tout en limitant les faux positifs. Son AUC élevée (0.968) confirme sa capacité à bien séparer les deux classes.

Globalement, les résultats montrent que certains modèles favorisent le recall tandis que d'autres offrent un meilleur compromis entre précision et rappel.

L'analyse des courbes Precision-Recall confirme les observations issues des métriques précédentes. Le modèle Random Forest se distingue clairement avec une aire sous la courbe (AP) élevée d'environ 0.882, ce qui traduit une excellente capacité à maintenir une précision élevée même lorsque le recall augmente. La régression logistique présente également de bonnes performances avec un AP de 0.732, montrant une bonne capacité de détection des fraudes, bien que la précision diminue progressivement lorsque le recall augmente.

Le modèle LinearSVC affiche une AUC-PR d'environ 0.755, indiquant un bon compromis entre précision et rappel, mais avec une certaine instabilité lorsque le recall devient élevé. Le modèle KNN présente des performances intermédiaires (AP = 0.557), avec une diminution progressive de la précision à mesure que le recall augmente. L'arbre de décision montre des performances plus limitées (AP = 0.396), ce qui traduit une capacité plus faible à maintenir une bonne précision.

Enfin, le modèle Naive Bayes présente les plus faibles performances (AP = 0.083), avec une chute rapide de la précision, ce qui indique une forte proportion de faux positifs lorsque le modèle tente d'augmenter le recall. Globalement, ces courbes confirment que les modèles les plus performants sont ceux capables de maintenir un bon niveau de précision tout en augmentant le recall.

6.3 Choix du meilleur modèle

Au vu des résultats obtenus, le modèle Random Forest apparaît comme le plus performant pour ce problème de détection de fraude. Contrairement aux autres modèles, il offre un excellent équilibre entre précision et recall, avec des valeurs proches de 0.85 pour ces deux métriques, ainsi qu'un F1-score également élevé.

Ce résultat signifie que le modèle est capable de détecter un grand nombre de fraudes tout en limitant les fausses alertes, ce qui est essentiel dans un contexte réel. En effet, un

modèle avec un recall élevé mais une précision faible génère trop de faux positifs, ce qui peut être coûteux et inefficace. À l'inverse, Random Forest parvient à maintenir un bon niveau sur les deux aspects.

De plus, son AUC élevée (0.968) confirme sa forte capacité de discrimination entre les classes. La matrice de confusion montre également un faible nombre d'erreurs, ce qui renforce la fiabilité du modèle.

Ainsi, Random Forest constitue le choix le plus adapté pour ce projet, grâce à sa robustesse, sa stabilité et ses performances équilibrées.

Chapitre 7

Conclusion

7.1 Résumé des résultats

Ce projet a permis d'appliquer plusieurs modèles de machine learning au problème de détection de fraude par carte bancaire. Les résultats obtenus montrent que les performances varient significativement selon les modèles utilisés.

Certains modèles comme la régression logistique, le SVM et Naive Bayes présentent un recall élevé, ce qui leur permet de détecter une grande partie des fraudes, mais au prix d'une précision faible. D'autres modèles comme KNN et l'arbre de décision offrent un meilleur équilibre entre précision et recall.

Le modèle Random Forest se distingue par les meilleures performances globales, avec un excellent compromis entre détection des fraudes et limitation des erreurs, ce qui en fait le modèle le plus adapté dans ce contexte.

7.2 Limites du projet

Malgré les résultats obtenus, ce projet présente certaines limites. Tout d'abord, les données utilisées sont anonymisées et issues d'une transformation PCA, ce qui empêche une interprétation directe des variables. Ensuite, le dataset ne couvre qu'une période limitée de deux jours, ce qui peut ne pas refléter l'ensemble des comportements réels.

Par ailleurs, certains modèles restent sensibles aux caractéristiques du dataset, notamment au déséquilibre initial et à la présence d'outliers. Enfin, les modèles utilisés n'ont pas fait l'objet d'une optimisation approfondie des hyperparamètres, ce qui pourrait améliorer davantage les performances.

7.3 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées pour approfondir ce travail. Il serait possible d'optimiser les hyperparamètres des modèles afin d'améliorer leurs performances. L'utilisation de techniques plus avancées, telles que les méthodes d'ensemble ou les modèles de deep learning, pourrait également être explorée.

Par ailleurs, l'intégration de nouvelles données sur une période plus longue permettrait d'obtenir des résultats plus représentatifs. Enfin, bien que l'analyse ait déjà intégré la courbe précision-rappel pour évaluer les performances, il serait pertinent d'explorer d'autres métriques adaptées aux datasets déséquilibrés ou d'affiner l'analyse des courbes existantes afin d'obtenir une évaluation encore plus fine des modèles.